

Web and Mobile System Algorithm Guide

How the Payroll, Attendance, POS, Inventory, Quotation, and Reporting features work from user input to database result.

Web: PHP + MySQL/MariaDB on XAMPP

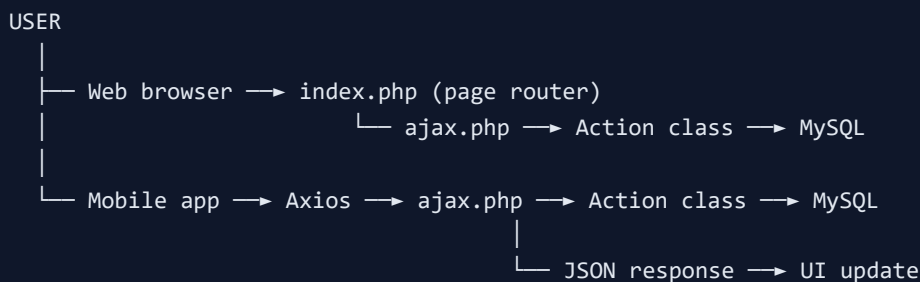
Mobile: React Native + TypeScript

API: `ajax.php?action=...`

Document version: 1.0 | September 2026

1. System Overview

The web and mobile applications use one shared business database. The web app is the administration and reporting interface. The mobile app is the operational interface for attendance, POS, quotations, inventory, and owner summaries.



Important design rule: the mobile screen may calculate a preview, but the server must recalculate important totals such as sales totals before saving.

2. Common Request Algorithm

1. Receive a page request or API request.
2. Identify the user through a web session or mobile login data.
3. Check role and branch/site scope.
4. Validate required values and normalize dates, numbers, and IDs.
5. Run the business operation in `admin_class.php`.
6. Read or write MySQL data using queries or prepared statements.
7. Return HTML for a page or JSON for a mobile/AJAX action.
8. Display success, validation, or error feedback to the user.

Layer	Web implementation	Mobile implementation
Interface	PHP pages, Bootstrap, jQuery	React Native screens and Zustand store
Transport	Browser GET/POST and jQuery AJAX	Axios GET/POST
Router	<code>index.php?page=...</code>	<code>AppNavigator.tsx</code>
Business logic	<code>admin_class.php</code>	Local UI logic plus server Action methods
Database	MySQL/MariaDB shared by both clients	

3. Login, Permission, and Role Routing

Web login

1. User submits username and password.
2. Server checks the user record and password hash.
3. On success, session values such as `login_id`, `login_role`, and branch/site are stored.
4. Protected pages reject missing or unauthorized sessions.
5. The router loads only the requested whitelisted page.

Mobile login and startup

1. `MainScreen` initializes the local database and checks microphone/storage permissions.
2. If permissions are missing, route to `PermissionPage`.
3. If a saved `userType` exists, route directly to the role landing screen.
4. Otherwise show login.
5. `checkLogin` calls `?action=mobile-login-check`.
6. On success, store user, role, branch/site, and employer values in `AsyncStorage`.

role 8 → POSDashboard | role 9 → PortalScreen | role 10 → OwnerDashboard |
other roles → Dashboard

Scope rule: administrators may see all records; branch/site users are filtered to their assigned branch/site before records are shown or changed.

4. Attendance and DTR Algorithm

4.1 Mobile/local DTR flow

1. Load employee, position, and department reference data from the API.
2. Store operational data locally for faster mobile use.
3. Create or edit DTR headers and detail logs locally.
4. Normalize each detail with employee code, date/time, type, logs, overtime, undertime, late, and hours.
5. Attach the current user and branch/site.
6. Push the complete DTR payload to `mobile-push-dtr`.

7. Server validates the payload, inserts the DTR header/details, and returns a result.

4.2 Biometric upload flow

1. Allow only an authorized role to upload a .dat file.
2. Read the file line by line and split fields by tabs or commas.
3. Reject incomplete lines or invalid date/time values.
4. Convert attendance codes: 0 = In, 1 = Out.
5. Resolve branch and device IDs from the file or logged-in user.
6. Calculate the covered date range.
7. Reject duplicate uploads with the same date range, branch, and device.
8. Insert the DTR header and all valid details in one database transaction.
9. Commit only if every insert succeeds; otherwise roll back.

```
DAT file → validate lines → parse employee code/date/type → resolve branch/device  
→ duplicate check → insert DTR header → insert details → commit/rollback
```

4.3 Approval and display

Uploaded records are assigned a status. The DTR server filters by status and user branch, supports pagination, joins employee/branch/user information, and returns DataTables JSON. Approved DTR details become payroll inputs.

5. Payroll Calculation Algorithm

Payroll calculation reads approved DTR details for the payroll date range, selected branches, and payroll type. Employees are grouped by employee ID before payroll items are generated.

1. Load payroll period, type, branch IDs, and contribution settings.
2. Read approved DTR details inside the period.
3. Filter by branch and weekly/monthly payroll type.
4. For each attendance detail, cap work hours at 8.
5. Convert hours to payable days: 8 hours = 1 day; otherwise hours ÷ 8, with the configured special handling for 4.5625 hours.
6. Accumulate regular hours, salary, present days, overtime, undertime, late minutes, and employee rates by employee.
7. Apply configured allowances, contributions, loans, deductions, taxes, and refunds.
8. Save or replace payroll items inside the calculation transaction.
9. Calculate gross pay, total deductions, refunds, and net pay.
10. Lock the payroll after review so final values cannot be changed casually.

Core formulas

$$\text{per_hour} = \text{per_day} \div 8$$

$$\text{per_minute} = \text{per_day} \div (8 \times 60)$$

```
regular_pay = work_hours × per_hour
```

```
overtime_pay = overtime_hours × overtime_rate
```

```
late_deduction = late_minutes × per_minute
```

```
holiday_pay = holiday_units × per_day (special holiday uses (per_day ÷ 8) × 2.4)
```

```
gross_pay = regular_pay + allowances + overtime_pay + holiday_pay - late_deduction
```

```
net_pay = gross_pay - total_deductions + total_refunds
```

Payroll output: one payroll item per employee, with attendance totals, earnings, deductions, gross salary, and net pay. Payslips and reports read these saved items.

6. POS Sales and Inventory Algorithm

6.1 Product loading

1. Mobile reads the saved branch ID.
2. Request products for that branch.
3. Display active products, price, quantity, and stock status.
4. Cashier adds products and quantities to a local cart.

6.2 Sale checkout

1. Reject an empty cart.
2. Server recalculates subtotal from each item price $\times$ quantity; do not trust the client total.
3. Clamp discount between zero and subtotal.
4. Compute total and change.
5. Begin a transaction.
6. Insert the sale header with invoice number, branch, cashier, subtotal, discount, total, payment, and change.
7. Insert every sale item.
8. Decrease each product stock using a non-negative update.
9. Read updated stock values.
10. Commit and return invoice/result; roll back on any exception.

```
subtotal =  $\Sigma$ (item.price  $\times$  item.quantity)
```

```
total = subtotal - min(max(discount, 0), subtotal)
```

```
change = max(payment - total, 0)
```

```
cart  $\rightarrow$  server recalculation  $\rightarrow$  transaction start  $\rightarrow$  sale header  
 $\rightarrow$  sale items  $\rightarrow$  decrease stock  $\rightarrow$  commit  $\rightarrow$  invoice + updated stock
```

6.3 Damage and stock adjustment

For a damage record, validate branch, cashier, item, and positive quantity. Insert the damage record and decrease stock in a transaction. For restocking, validate product and positive quantity, then increase stock.

7. Quotation Algorithm

1. Load products and previous quotations for the current branch.
2. Validate selected product and positive width/height.
3. Convert the entered unit to meters and square meters.
4. Calculate glass area cost and aluminum perimeter cost.
5. Add optional installation, design, and measurement labor.
6. Apply manual line-price overrides, if entered.

7. Sum line items to produce subtotal.
8. Apply fixed or percentage discount, limited to subtotal.
9. Calculate tax on the discounted subtotal.
10. Calculate final total.
11. Add the configured quotation item to the cart.
12. Save the cart through the API and refresh quotation history.

Quotation formulas

```
area_m2 = width × height × unit_area_conversion
```

```
perimeter_m = 2 × (width + height) × unit_length_conversion
```

```
glass_cost = price_per_m2 × area_m2 × design/type_multiplier
```

```
aluminum_cost = price_per_m × perimeter_m × profile_multiplier
```

```
subtotal = Σ(line_total)
```

```
discount = fixed amount OR subtotal × percentage ÷ 100
```

```
tax = (subtotal - discount) × tax_percentage ÷ 100
```

```
total = subtotal - discount + tax
```

Unit	Length conversion to meters	Area conversion to square meters
MM	0.001	0.000001
CM	0.01	0.0001
IN	0.0254	0.00064516
Ft	0.3048	0.092903
M	1	1

Typical labor defaults are ₱500 per panel for installation, ₱1,500 for design customization, and ₱300 for measurement. The current mobile quotation screen recalculates its summary with `useMemo` whenever inputs change.

8. Reporting and Owner Dashboard Algorithm

1. Mobile owner screens request report actions through Axios.
2. Server verifies the action and queries aggregate counts/totals.
3. Results are returned as JSON.
4. The app maps each result to a report card.
5. Selecting a card loads branch-aware report details.

Report	Typical output
Sales	Transaction count and sales total
Inventory	Product count, stock, and low-stock state
Attendance	Attendance count/rate and DTR summaries
Payroll	Active employee/payroll count
Quotations	Quotation count and total quoted value

9. Error Handling and Data Integrity

- Validate input before database writes.
- Use role and branch/site filters on protected data.
- Use transactions for multi-table operations: DTR import, sales, damage, and payroll generation.
- Rollback on failure so partial records are not treated as complete operations.
- Return user-safe error messages; log technical database details on the server.
- Use loading/success/error states in the mobile Zustand store so users know whether an operation completed.
- Refresh lists after successful saves to keep the UI consistent with the database.

Maintenance note: the algorithm guide describes the current implementation. If rates, roles, table names, or API actions change, update this document together with the related source code and tests.

10. Main Source References

Area	Web files	Mobile files
Routing/auth	index.php, ajax.php, admin_class.php	src/AppNavigator.tsx, src/screens/MainScreen.tsx, src/store/globalState.ts
DTR	admin_class.php, dtr-server.php	globalState.ts, DTR screens, biometric utilities
Payroll	admin_class.php, payroll_items.php	Payroll/DTR screens and report screens
POS	admin_class.php, POS pages and JS	src/screens/pos/*, src/utils/posService.ts
Quotation	Quotation API methods in admin_class.php	POSQuotationTab.tsx, quotationCalculator.ts

Prepared from the current source and project documentation in `c:\xampp\htdocs\payroll` and `c:\xampp\htdocs\mobile`. This guide is intended for system explanation, onboarding, and project presentation.